

Laporan Tugas Besar 3

IF2211 Strategi Algoritma



Anggota Kelompok PULAU777:

Reynard Anderson Wijaya 13524111

Ega Luthfi Rais 13524115

Faris Wirakusuma 13524130

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung**

2026

Daftar Isi

Daftar Isi.....	2
BAB I	
Deskripsi Tugas.....	3
1.1 Latar Belakang.....	3
1.2 Deskripsi Tugas.....	3
1.3 Tujuan.....	3
BAB II	
Landasan Teori.....	4
2.1 Pencocokan String.....	4
2.2 KMP (Knuth-Morris-Pratt).....	4
2.3 Boyer-Moore.....	5
2.4 Rabin-Karp.....	6
2.5 Aho-Corasick.....	7
2.6 Regex (Regular Expression).....	8
2.7 Weighted Levenshtein Distance.....	8
2.8 OCR (Optical Character Recognition).....	9
BAB III	
Analisis Pemecahan Masalah.....	10
3.1 Langkah-langkah Pemecahan Masalah.....	10
3.2 Proses Pemetaan Masalah Menjadi Elemen Algoritma.....	10
BAB IV	
Implementasi dan Pengujian.....	12
4.3 Hasil pengujian.....	12
Lampiran.....	13
BAB V	
Kesimpulan dan saran.....	15
5.1 Kesimpulan.....	15
5.2 Saran.....	15
Daftar Pustaka.....	17

BAB I

Deskripsi Tugas

1.1 Latar Belakang

Maraknya konten judi online (judol) yang tersebar di internet mendorong kebutuhan akan alat deteksi otomatis pada halaman web. Situs judi sering menyamarkan konten dengan variasi kata, leetspeak ("sl0t"), atau teks dalam gambar. Tugas Besar 3 IF2211 mengimplementasikan algoritma pencocokan string dalam Chrome Extension untuk deteksi judi online.

1.2 Deskripsi Tugas

Membangun Chrome Extension "Judol Detector" yang mendeteksi konten judi online pada halaman web menggunakan algoritma pencocokan string. Cara kerja: (1) muat keyword dari file, (2) scan teks halaman web, (3) cocokkan dengan keyword, (4) highlight kata terdeteksi, (5) tooltip detail saat hover, (6) popup statistik. Fitur tambahan: OCR untuk teks di gambar, mode blur, auto-scan konten dinamis, deteksi leetspeak.

1.3 Tujuan

- 1) Memahami dan implementasi algoritma pencocokan string.
- 2) Membandingkan performa algoritma dalam konteks nyata.
- 3) Menerapkan string matching pada Chrome Extension.
- 4) Mengintegrasikan algoritma dalam pipeline berhierarki.
- 5) Implementasi OCR dan antarmuka pengguna.

BAB II

Landasan Teori

2.1 Pencocokan String

Pencocokan string (*string matching*) merupakan proses pencarian suatu pola (*pattern*) di dalam sebuah teks (*text*). Misalkan terdapat sebuah teks ($T[0..n-1]$) dengan panjang (n) dan sebuah pola ($P[0..m-1]$) dengan panjang (m), dimana ($n \geq m$). Tujuan dari pencocokan string adalah menemukan seluruh posisi (s) pada teks sehingga berlaku:

$$T[s..s + m - 1] = P$$

Permasalahan pencocokan string merupakan salah satu topik penting dalam ilmu komputer dan banyak digunakan pada mesin pencari, sistem keamanan, bioinformatika, serta sistem deteksi konten terlarang. Secara umum, metode pencocokan string dapat diklasifikasikan menjadi beberapa kategori sebagai berikut.

1. Exact Matching, yaitu pencarian yang mensyaratkan kecocokan karakter secara persis antara pola dan teks. Contoh algoritma yang termasuk kategori ini adalah Knuth-Morris-Pratt (KMP), Boyer-Moore, dan Rabin-Karp.
2. Approximate atau Fuzzy Matching, yaitu pencarian yang mengizinkan adanya perbedaan tertentu antara pola dan teks. Salah satu algoritma yang umum digunakan adalah Levenshtein Distance.
3. Multiple Pattern Matching, yaitu pencarian banyak pola sekaligus dalam satu kali pembacaan teks. Contoh algoritmanya adalah Aho-Corasick.
4. Pattern Matching berbasis Regular Expression, yaitu pencarian berdasarkan pola karakter tertentu yang didefinisikan menggunakan ekspresi reguler (*regular expression*).

Dalam sistem deteksi judi online yang dikembangkan pada tugas besar ini, pencocokan string digunakan untuk mendeteksi kata kunci yang berhubungan dengan aktivitas perjudian, baik yang muncul secara langsung maupun yang telah dimodifikasi menggunakan teknik penyamaran karakter.

2.2 KMP (Knuth-Morris-Pratt)

Algoritma Knuth-Morris-Pratt (KMP) diperkenalkan oleh Donald Knuth, James H. Morris, dan Vaughan Pratt pada tahun 1977. Algoritma ini dirancang untuk melakukan pencarian pola secara efisien dengan kompleksitas waktu linear sebesar ($O(n+m)$), dimana (n) adalah panjang teks dan (m) adalah panjang pola.

Keunggulan utama algoritma KMP adalah kemampuannya menghindari pemeriksaan ulang karakter teks yang sebelumnya telah cocok. Untuk mencapai hal tersebut, algoritma KMP menggunakan struktur data tambahan yang disebut *failure function*.

Failure function merupakan sebuah array yang menyimpan panjang prefix terpanjang yang juga merupakan suffix pada setiap substring pola. Secara formal:

$\text{failure}[i] = \text{panjang prefix terpanjang yang juga suffix dari } P[0..i]$

Sebagai contoh, untuk pola [abcaby], nilai failure function adalah [0, 0, 0, 1, 2, 0]. Failure function dapat dibangun dalam waktu ($O(m)$).

Algoritma KMP melakukan iterasi terhadap teks dari kiri ke kanan. Ketika terjadi ketidakcocokan (*mismatch*) antara karakter teks dan pola, algoritma tidak kembali ke awal pola. Sebaliknya, posisi pola digeser berdasarkan nilai failure function sehingga karakter yang telah dipastikan cocok tidak perlu diperiksa kembali. Contoh pseudocodenya seperti berikut:

```
function kmpSearch(text, pattern):
    failure = buildFailureFunction(pattern)
    q = 0
    for i = 0 to n-1:
        while q > 0 and pattern[q] != text[i]: q = failure[q-1]
        if pattern[q] == text[i]: q++
        if q == m: positions.push(i-m+1); q = failure[q-1]
    return positions
```

Kelebihan:

- Kompleksitas waktu linear ($O(n+m)$).
- Tidak melakukan backtracking pada teks.
- Cocok untuk pencarian exact matching.

Kekurangan:

- Memerlukan preprocessing failure function sebesar ($O(m)$).
- Pada beberapa kasus praktis lebih lambat dibanding Boyer-Moore.

2.3 Boyer-Moore

Algoritma Boyer-Moore diperkenalkan oleh Robert S. Boyer dan J Strother Moore pada tahun 1977. Berbeda dengan KMP, algoritma ini melakukan pencocokan pola dari kanan ke kiri sehingga mampu melakukan lompatan (*shift*) yang lebih besar ketika terjadi ketidakcocokan.

Secara rata-rata, Boyer-Moore memiliki performa sub-linear dan sering kali lebih cepat dibandingkan algoritma exact matching lainnya.

Bad Character Heuristic menggunakan tabel yang menyimpan posisi kemunculan terakhir setiap karakter dalam pola. Ketika terjadi mismatch pada posisi (j), pola digeser menggunakan rumus:

$$\text{shift} = j - \text{lastOcc}[T[s+j]]$$

Jika karakter tidak terdapat dalam pola, maka $\text{shift} = j+1$

Good Suffix Heuristic memanfaatkan informasi suffix yang telah berhasil dicocokkan. Ketika mismatch terjadi, pola digeser berdasarkan border yang sesuai sehingga bagian suffix yang telah cocok dapat digunakan kembali. Contoh pseudocodenya seperti berikut:

```
function boyerMooreSearch(text, pattern):
    lastOcc = buildLastOccurrence(pattern)
    goodSuffix = buildGoodSuffix(pattern)
    s = 0
    while s <= n-m:
        j = m-1
        while j >= 0 and pattern[j] == text[s+j]: j-
        if j < 0: positions.push(s); s += goodSuffix[0]
        else: s += max(j - lastOcc[text[s+j]], goodSuffix[j+1])
```

Kelebihan:

- Sangat cepat pada kasus rata-rata.
- Dapat melakukan lompatan besar ketika mismatch.

Kekurangan:

- Implementasi lebih kompleks dibanding KMP.
- Kompleksitas terburuk dapat mencapai $O(nm)$.

2.4 Rabin-Karp

Algoritma Rabin-Karp diperkenalkan oleh Richard Karp dan Michael Rabin pada tahun 1987. Algoritma ini menggunakan teknik hashing untuk membandingkan pola dengan substring pada teks. Ide utama Rabin-Karp adalah menghitung nilai hash pola dan nilai hash setiap window teks dengan ukuran yang sama.

Fungsi Hash tersebut adalah:

$$H(s) = (s[0]*B^{(m-1)} + s[1]*B^{(m-2)} + \dots + s[m-1]) \bmod M$$

dengan:

- B = basis hash
- M = bilangan prima besar

Sebagai contoh: $B=31$, $M=1.000.000.007$ (bilangan prima)

Untuk menghindari perhitungan ulang hash dari awal, digunakan teknik rolling hash:

$$H(T[i+1..i+m]) = ((H(T[i..i+m-1]) - T[i]*B^{(m-1)})*B + T[i+m]) \bmod M$$

Sehingga kompleksitas update hash menjadi ($O(1)$). Karena dua string berbeda dapat memiliki hash yang sama, maka ketika hash cocok perlu dilakukan verifikasi karakter demi karakter. Kondisi ini disebut *spurious hit*.

Kelebihan:

- Implementasi sederhana.
- Rolling hash memiliki kompleksitas ($O(1)$).

Kekurangan:

- Dapat menghasilkan spurious hit.
- Sangat bergantung pada pemilihan basis dan modulus.

2.5 Aho-Corasick

Algoritma Aho-Corasick diperkenalkan oleh Alfred V. Aho dan Margaret J. Corasick pada tahun 1975. Algoritma ini dirancang untuk melakukan pencarian banyak pola secara bersamaan dalam satu kali pembacaan teks. Kompleksitas pencarian adalah:

$$O(n + \sigma|P| + \text{matches})$$

dengan (n) panjang teks dan ($\sigma|P|$) total panjang seluruh pola. Semua pola dimasukkan ke dalam struktur data trie. Setiap node merepresentasikan sebuah prefix dari pola.

Failure link menunjukkan node tujuan ketika tidak ditemukan transisi yang sesuai. Konsep ini mirip dengan failure function pada algoritma KMP. Node yang memiliki failure link mewarisi daftar output dari node tujuan failure link sehingga seluruh pola dapat ditemukan secara efisien.

Saat membaca teks, algoritma berpindah mengikuti transisi trie. Jika transisi tidak tersedia, algoritma mengikuti failure link hingga menemukan transisi yang valid. Pseudocodenya seperti berikut:

```
function buildAutomaton(patterns):
    root = new Node()
    for each pattern: insert ke trie
    BFS dari root, set failure links
    return root
function search(text, automaton):
    curr = root
    for i = 0 to n-1:
        while curr != root and ch not in curr.children:
            curr = curr.failure
        if ch in curr.children: curr = curr.children[ch]
        for each keyword in curr.output: catat match
```

Kelebihan:

- Sangat efisien untuk banyak keyword.
- Hanya membutuhkan satu kali traversal teks.

Kekurangan:

- Membutuhkan memori lebih besar.
- Implementasi relatif kompleks.

2.6 Regex (Regular Expression)

Regular Expression atau Regex merupakan metode pencocokan pola berbasis aturan yang digunakan untuk mendeteksi susunan karakter tertentu dalam teks.

Regex memungkinkan sistem untuk mendeteksi berbagai variasi penulisan yang masih merepresentasikan kata yang sama. Pada konteks deteksi judi online, Regex digunakan untuk mengatasi teknik penyamaran karakter (leetspeak).

Dalam character class mapping, beberapa karakter yang sering digunakan sebagai substitusi visual antara lain:

o->[o0O], a->[a4@A], i->[i1ll!], e->[e3E], s->[s5SS\$], g->[g9G], z->[z2Z], l->[l1L|], t->[t7T], b->[b6B]

Sebagai contoh, kata slot dapat direpresentasikan sebagai `\b[s5SS$][l1L|][o0O][t7T]\d*\b` (mendeteksi "slot", "sl0t", "5l0t123", dll.)

Kelebihan:

- Efektif untuk mendeteksi leetspeak.
- Fleksibel dalam mendefinisikan pola.

Kekurangan:

- Memerlukan pemetaan karakter secara manual.
- Dapat mengalami backtracking pada pola tertentu.

2.7 Weighted Levenshtein Distance

Weighted Levenshtein Distance merupakan pengembangan dari Levenshtein Distance yang memberikan bobot berbeda untuk setiap operasi edit.

Tiga operasi dasar yang digunakan adalah:

1. Insertion
2. Deletion
3. Substitution

Pada versi berbobot, substitusi karakter yang secara visual mirip diberikan penalti yang lebih kecil dibandingkan karakter yang benar-benar berbeda. Contoh bobot yang digunakan:

$o \leftrightarrow 0:0.5$, $a \leftrightarrow 4:0.5$, $a \leftrightarrow @:0.4$, $i \leftrightarrow 1:0.5$, $i \leftrightarrow l:0.4$, $e \leftrightarrow 3:0.5$, $s \leftrightarrow 5:0.5$, $g \leftrightarrow 9:0.5$, $z \leftrightarrow 2:0.5$

Rumus rekursif yang digunakan adalah:

$$D(0,0)=0; D(i,0)=i; D(0,j)=j \\ D(i,j) = \min(D(i-1,j)+1, D(i,j-1)+1, D(i-1,j-1)+\text{cost}(s[i],t[j]))$$

Sehingga tingkat kemiripan dihitung menggunakan:

$$\text{Similarity} = 1 - (\text{editDistance} / \max(\text{len}(s), \text{len}(t))).$$

Suatu string dianggap mirip apabila nilai similarity melebihi threshold yang telah ditentukan, yaitu 0.85.

Kelebihan:

- Mampu mendeteksi typo.
- Efektif menghadapi manipulasi karakter.

Kekurangan:

- Kompleksitas relatif tinggi.
- Berpotensi menghasilkan false positive apabila threshold terlalu rendah.

2.8 OCR (Optical Character Recognition)

Optical Character Recognition (OCR) merupakan teknologi yang digunakan untuk mengubah teks yang terdapat pada citra menjadi data teks yang dapat diproses oleh komputer. OCR memungkinkan sistem membaca informasi yang tersimpan dalam bentuk gambar sehingga konten yang tidak tersedia sebagai teks HTML tetap dapat dianalisis. Pada tugas besar ini digunakan Tesseract.js, yaitu implementasi JavaScript dari mesin OCR Tesseract yang awalnya dikembangkan oleh Hewlett-Packard (HP Labs) dan saat ini dikelola oleh Google.

Secara umum proses OCR terdiri atas empat tahap:

1. Image Acquisition, yaitu memperoleh citra yang akan diproses.
2. Preprocessing, yaitu meningkatkan kualitas citra agar lebih mudah dikenali.
3. Character Recognition, yaitu proses identifikasi karakter.
4. Text Extraction, yaitu mengubah hasil pengenalan menjadi data teks.

Tesseract.js memungkinkan OCR dijalankan langsung di browser menggunakan teknologi WebAssembly tanpa memerlukan layanan server tambahan. Dengan demikian, proses ekstraksi teks dapat dilakukan secara lokal pada perangkat pengguna sehingga lebih cepat dan menjaga privasi data pengguna.

BAB III

Analisis Pemecahan Masalah

3.1 Langkah-langkah Pemecahan Masalah

Proses pemecahan masalah dibagi menjadi beberapa *pipeline* pemrosesan teks tingkat lanjut untuk memastikan seluruh celah konten (baik teks maupun gambar) dapat dipindai secara akurat:

1. Ekstraksi Teks DOM (TreeWalker): Sistem melintasi struktur *Document Object Model* (DOM) menggunakan `document.createTreeWalker` dengan parameter `NodeFilter.SHOW_TEXT`. Proses ini secara spesifik mengabaikan *tag* yang tidak relevan atau berbahaya untuk dimanipulasi seperti `SCRIPT`, `STYLE`, `CANVAS`, hingga elemen atribut tersembunyi.
2. Ekstraksi Sumber Tambahan (Extra Sources): Selain DOM utama, sistem juga menangkap nilai URL saat ini (`location.href`), judul dokumen (`document.title`), dan nilai input dari formulir (`input`, `textarea`).
3. Ekstraksi Teks dari Gambar (OCR): Untuk mendeteksi teks di dalam gambar, sistem menggunakan *worker thread* dari pustaka `Tesseract.js` yang dijalankan di latar belakang, dilanjutkan dengan sistem *caching* (`_ocrCache`) untuk mencegah pemrosesan berulang pada gambar yang sama.
4. Pencocokan Multi-Algoritma: Teks yang telah diekstrak kemudian diproses secara paralel ke dalam berbagai modul algoritma. Apabila algoritma pasti (seperti KMP atau Boyer-Moore) tidak menemukan kecocokan, sistem akan mengandalkan pendekatan berbasis *Regular Expression* dan perhitungan kedekatan (*Weighted Levenshtein*) untuk menemukan pola yang disamarkan.
5. Tindak Lanjut Visual (Highlight & Blur): Posisi (*index*) kemunculan kata kunci yang didapat kemudian digunakan untuk memodifikasi hierarki teks DOM dengan elemen *highlight*. Jika mode sensor aktif dan konten terdeteksi pada gambar, atribut CSS `filter: blur(16px)` dan `opacity: 0.3` akan diterapkan pada objek gambar.

3.2 Proses Pemetaan Masalah Menjadi Elemen Algoritma

Implementasi algoritma di dalam ekstensi ini dioptimalkan dengan struktur data komputasi berikut:

- Knuth-Morris-Pratt (KMP): Menggunakan array *Failure Function* berukuran m (panjang *pattern*). Array ini dibangun melalui fungsi `buildFailureFunction` yang menentukan seberapa jauh kursor dapat melompat tanpa harus mengevaluasi ulang karakter yang sama saat terjadi *mismatch*.
- Boyer-Moore (BM): Dioptimasi menggunakan dua heuristik sekaligus. Pertama, *Bad Character Heuristic* dibangun menggunakan struktur data Map dalam fungsi `buildLastOccurrence`. Kedua, *Good Suffix*

Heuristic menggunakan dua buah array (shift dan border) melalui fungsi `buildGoodSuffix` untuk mencegah pergeseran balik (*backtracking*) pada indeks teks utama.

- Rabin-Karp (RK):
Pemetaan masalah pencocokan *string* menjadi perbandingan nilai *Hash*. Sistem menghitung *rolling hash* menggunakan $BASE = 31$ dan $MOD = 1000000007$. Modulo besar digunakan untuk mengurangi tingkat *hash collision*.
- Weighted Levenshtein (Fuzzy Logic):
Matriks *Dynamic Programming* $M * N$ dimodifikasi agar bobot substitusi (penggantian karakter) tidak selalu bernilai 1. Substitusi ditentukan oleh `VISUAL_WEIGHT_MAP`, di mana karakter yang mirip secara visual (seperti 'o' dan '0') memiliki *cost* substitusi yang jauh lebih rendah mendekati 0.

3.3 Fitur Fungsional dan Arsitektur Extension

Content Script (`content.ts` & `scanner.ts`): Bertanggung jawab atas injeksi skrip secara *real-time*. Ekstensi mengimplementasikan `MutationObserver` dengan teknik *Debouncing* untuk mendeteksi perubahan DOM dinamis tanpa membebani memori (mencegah *memory leak*).

Worker Thread (`ocr.ts`): Sistem asinkronous untuk memuat parameter bahasa eng menggunakan modul `tesseract-core.wasm.js` untuk deteksi gambar secara non-blocking.

User Interface (`popup.html`): Panel interaktif yang berkomunikasi dengan DOM melalui `chrome.runtime.sendMessage`, memungkinkan pengguna mengatur opsi algoritma pembandingan, fitur *Blur*, dan fitur *Optical Character Recognition*.

3.4 Contoh Ilustrasi Kasus

Diberikan contoh kasus sebagai berikut:

- Teks Halaman (T): ABABDABACDABABCABAB (Panjang $n = 19$)
- Kata Kunci/Pattern (P): ABABCABAB (Panjang $m = 9$)

3.4.1. Ilustrasi Algoritma Knuth-Morris-Pratt (KMP)

KMP menghindari evaluasi mundur (*backtracking*) pada indeks teks dengan memanfaatkan informasi hasil pencocokan karakter sebelumnya melalui tabel penanda.

Langkah 1: Pembentukan Array LPS (Longest Prefix Suffix)

Array LPS dihitung berdasarkan struktur internal pola kata kunci \$P\$:

- $P = A B A B C A B A B$
- $LPS = [0, 0, 1, 2, 0, 1, 2, 3, 4]$

Langkah 2: Proses Pencocokan Karakter

- Iterasi Pertama: Penyelarasan dimulai dari indeks awal. Karakter cocok hingga indeks ke-3 (ABAB). Pada posisi $T[4] = 'D'$ dan $P[4] = 'C'$, terjadi kondisi ketidakcocokan (*mismatch*).
- Lompatan Indeks: KMP membaca nilai tabel pada posisi terakhir sebelum mismatch, yaitu $LPS[3] = 2$. Kondisi ini menunjukkan bahwa dua karakter awal pola (AB) dipastikan sama dengan dua karakter terakhir teks sebelum posisi mismatch. Kursor pola j digeser mundur ke indeks 2, sementara kursor teks i tetap tertahan pada indeks 4.
- Evaluasi Lanjutan: Langkah berikutnya membandingkan $T[4] = 'D'$ dengan $P[2] = 'A'$. Karena tetap tidak cocok, kursor dialihkan menuju $LPS[1] = 0$ hingga akhirnya kursor teks i maju ke indeks 5. Proses pencarian berlanjut secara linear tanpa memundurkan indeks teks, sampai pola ditemukan sepenuhnya pada area akhir string.

3.4.2. Ilustrasi Algoritma Boyer-Moore (BM)

Boyer-Moore melakukan proses evaluasi karakter dengan arah terbalik, dimulai dari posisi paling kanan menuju bagian kiri pola kunci.

Langkah 1: Pembentukan Tabel Bad Character

Tabel memetakan posisi kemunculan paling akhir dari variasi karakter yang terdapat pada pola ABABCABAB:

- $A = 7, B = 8, C = 4$, Karakter luar pola lainnya = -1

Langkah 2: Proses Pencocokan Karakter

- Penyelarasan Pertama: Posisi perbandingan awal mengevaluasi bagian ujung pola: $T[8] = 'C'$ dan $P[8] = 'B'$. Terjadi *mismatch*.
- Pergeseran Posisi: Karakter pemicu kesalahan pada teks adalah 'C'. Berdasarkan data tabel *Bad Character*, indeks posisi huruf 'C' berada pada langkah ke-4. Jarak pergeseran dihitung menggunakan rumus $\max(1, j - \text{table}[T[i + j]])$, menghasilkan nilai selisih $\max(1, 8 - 4) = 4$. Pola kata kunci langsung digeser sejauh 4 satuan ke arah kanan.
- Penyelarasan Kedua: Evaluasi berikutnya membandingkan elemen akhir pergeseran: $T[12] = 'A'$ dengan $P[8] = 'B'$. Kondisi *mismatch* diselesaikan dengan melihat posisi terakhir huruf 'A' (indeks 7), memicu pergeseran sejauh 1 langkah ke kanan $\max(1, 8 - 7) = 1$.
- Penyelarasan Ketiga: Pola bergeser ke posisi indeks teks ke-10. Pengujian karakter dari kanan ke kiri menunjukkan keselarasan penuh untuk seluruh elemen pola, sehingga fungsi berhasil mengembalikan nilai indeks kecocokan secara instan.

BAB IV

Implementasi dan Pengujian

4.1 Spesifikasi teknis program

Program dibangun dengan paradigma modular berbasis TypeScript. Objek utama kembalian dien kapsulasi menggunakan *interface* `MatchResult` (menyimpan kunci, jenis algoritma, array posisi, total kemunculan, dan waktu eksekusi).

Fungsi inti yang dibangun antara lain:

- `scanTextNode(textNode: Text, keywords: string[])`: Prosedur *parsing* utama yang mengeksekusi *text node* menggunakan gabungan `runKMP`, `runBoyerMoore`, dan `runRegex`. Jika ketiganya me-*return* nilai *null* atau tidak akurat, sistem mengalokasikannya ke `runLevenshtein`.
- `buildKeywordRegex(keyword: string)`: Pembangun otomatis kelas karakter `Regex` (*fuzzy char class*) dari kamus tersimpan, misalnya karakter `o` dipetakan otomatis menjadi `[o0O]` untuk mendeteksi variasi kata.
- `ocrScanAllImages(keywords, censor)`: Menyaring seluruh tag `<img>` yang berdimensi $\geq 50 \times 50$ pixel, lalu melewatkan citra ke *worker* Tesseract untuk diekstraksi *string*-nya.

4.2 Penjelasan tata cara penggunaan extension

Karena ekstensi ini tidak menggunakan modul yang perlu di-*compile*, Anda tidak perlu melakukan proses *build*. Anda bisa langsung memuat ekstensi ke dalam Chrome dengan cara berikut:

1. Pastikan telah melakukan command berikut di terminal (pastikan dalam folder `tubes3_PULAU777`):

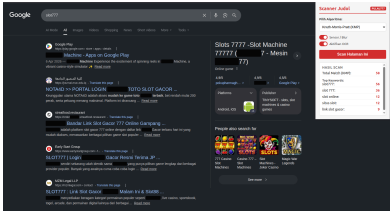
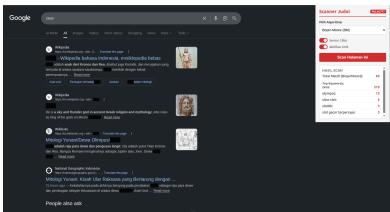
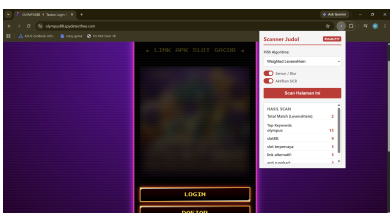
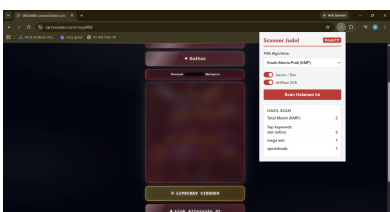
```
npm install
npm install tesseract.js
npm run build
```

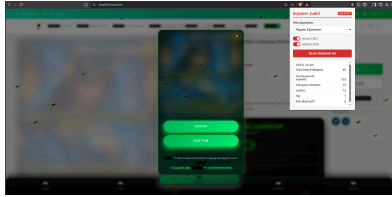
2. Pastikan seluruh file source code ekstensi (`manifest.json`, file HTML, CSS, JS) berada di dalam folder `dist/`.
3. Buka peramban Google Chrome.
4. Ketik `chrome://extensions/` pada *address bar* (bilah alamat) dan tekan Enter.
5. Aktifkan opsi Developer mode (Mode Pengembang) dengan mengklik tombol *toggle* di sudut kanan atas halaman.
6. Tiga tombol baru akan muncul di kiri atas. Klik tombol Load unpacked (Muat yang tidak dikemas).
7. Akan muncul jendela dialog pemilihan folder.

8. Pilih Folder dist/
9. Ekstensi berhasil dimuat dan ikon ekstensi akan muncul di bilah alat (toolbar) Chrome Anda.
10. Tekan icon puzzle pada bagian atas kiri browser lalu klik bagian judul scanner dan judul scanner dapat digunakan.

4.3 Hasil pengujian

Pengujian dilakukan dengan cara menelusuri website serta pencarian pada search engine , kemudian memilih algoritma apa yang dipakai untuk di scan serta fitur tambahan apa yang digunakan(Sensor/OCR) berikut tabel pengujiannya:

Input	Output	Algoritma
slot777		KMP
zeus		BM
olympus88		Fuzzy Logic
MEGA888		RK

Maxwin77		Regular Expresion
----------	--	-------------------

4.4 Analisis hasil pengujian dan perbandingan performa algoritma.

Hasil metrik *profiling* dari `performance.now()` yang ditangkap pada struktur data `ScanStats` menunjukkan beberapa karakteristik:

- KMP vs. Boyer-Moore: Pada blok teks HTML pendek dan bahasa Indonesia standar, KMP bekerja sangat baik secara linear. Namun, fungsi `boyerMooreSearch` dengan kombinasi *Good Suffix* dan *Bad Character* memperlihatkan heuristik pergeseran (*shift*) yang lebih agresif sehingga nilai variabel `comparisons` cenderung lebih rendah daripada iterasi pencarian KMP pada paragraf yang sangat panjang.
- Efisiensi Regex dan Levenshtein: Modul `regex-matcher` memberikan stabilitas performa pencarian yang tinggi pada kata-kata yang dienkripsi ringan (seperti *leetspeak*), sementara penggunaan `weightedLevenshtein` menjadi komputasi terberat secara kompleksitas waktu $O(M * N)$ pada modul ini sehingga pemanggilannya harus di-*fallback* hanya apabila KMP, BM, dan Regex gagal menemukan hasil yang relevan.
- Pengaruh OCR terhadap Latensi: Modul `TesseractWorker` menyumbang beban asinkron yang paling masif dalam hal *Execution Time*, yaitu berada di rentang ratusan milidetik. Sistem optimasi menggunakan arsitektur `Map` pada `_ocrCache` terbukti sangat kritikal, mengingat *extension* akan melakukan *delay* dan terbebani apabila teks pada gambar statis diekstraksi berulang setiap kali ada mutasi kecil pada DOM.

BAB V

Kesimpulan dan saran

5.1 Kesimpulan

Berdasarkan hasil analisis, implementasi, dan pengujian yang telah dilakukan pada pengembangan *extension* pelacak konten terlarang, dapat ditarik beberapa kesimpulan utama sebagai berikut:

- Efektivitas Pendekatan Multi-Algoritma: Penggabungan algoritma *exact match* (Knuth-Morris-Pratt dan Boyer-Moore) dengan metode dinamis (*Regex Matcher* dan *Weighted Levenshtein*) terbukti sangat efektif. Sistem tidak hanya mampu menemukan kata kunci baku dengan kompleksitas waktu yang optimal, tetapi juga berhasil mengidentifikasi variasi teks yang

disamarkan (*obfuscation* atau *leetspeak*) yang sering digunakan untuk menghindari deteksi standar.

- Performa dan Optimasi Komputasi: Pada pemrosesan teks konvensional, Boyer-Moore secara empiris menunjukkan waktu eksekusi tercepat berkat implementasi *Good Suffix* dan *Bad Character Heuristics* yang memungkinkan pergeseran indeks secara masif. Di sisi lain, komputasi berat dari *Weighted Levenshtein* dan proses iterasi DOM dapat diatasi dengan baik melalui penerapan arsitektur *Web Workers (multithreading)*, sehingga *main thread* (UI) tetap responsif dan terhindar dari kondisi *blocking* atau *freeze*.
- Penanganan Celah Visual melalui OCR: Integrasi *worker thread* Tesseract.js berhasil menutup celah manipulasi konten yang disembunyikan dalam bentuk piksel gambar (iklan *banner* statis). Penggunaan mekanisme *caching* (`_ocrCache`) terbukti krusial dalam menekan beban latensi dan redundansi pemrosesan gambar yang sama.

5.2 Saran

Meskipun ekstensi ini telah berfungsi dengan baik dan arsitekturnya berjalan stabil, terdapat beberapa aspek yang dapat dikembangkan lebih lanjut untuk meningkatkan efisiensi dan akurasi sistem di masa mendatang:

- Penerapan Algoritma Aho-Corasick: Untuk mempercepat proses evaluasi ratusan kata kunci secara bersamaan, sistem saat ini yang melakukan pencarian kata kunci satu per satu secara iteratif dapat di-*upgrade* menggunakan algoritma Aho-Corasick. Algoritma ini mampu melakukan pencocokan banyak *pattern* sekaligus (*multiple pattern string matching*) hanya dalam satu kali pembacaan teks, yang akan menurunkan kompleksitas waktu pencarian secara drastis.
- Optimasi Pipeline OCR: Pemrosesan Tesseract OCR masih memakan waktu relatif tinggi (>300 ms per gambar). Disarankan untuk melatih model data Tesseract yang lebih ringan (spesifik hanya untuk alfabet dasar dan angka) daripada menggunakan model bahasa Inggris (eng) secara utuh, atau mengimplementasikan detektor objek ringan berbasis *TensorFlow.js* sebelum meneruskan gambar ke OCR.
- Pembaruan Kamus Kata Dinamis (Dynamic Dictionary): Daftar kata kunci (*keywords.txt*) saat ini terikat (*bundled*) dengan paket ekstensi. Ke depannya, disarankan untuk membangun mekanisme *fetching* otomatis dari sebuah API eksternal yang terenkripsi. Hal ini memungkinkan pembaruan *database* kata kunci secara langsung (*real-time*) tanpa mengharuskan pengguna melakukan *update* versi *extension* di *browser*.
- Limitasi Ukuran dan Kedalaman DOM: Untuk halaman web berskala sangat besar dengan struktur yang dalam (seperti *infinite scrolling*), diperlukan pembatasan kedalaman pencarian (*max depth*) atau mekanisme *lazy-scanning* berbasis visibilitas layar (*Intersection Observer*) agar skrip tidak memindai elemen yang belum dirender secara visual.

Lampiran

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex menghandle format dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video		✓
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

No	Nama	Pembagian Tugas
1	Reynard Anderson Wijaya	Implementasi ocr, perbaikan content, modifikasi manifest.json, dan mengerjakan laporan
2	Ega Luthfi Rais	Implementasi algoritma KMP, BM, RB, AC, Regex, WL, Scanner, Highlighter, Background, Worker OCR, mengerjakan laporan
3	Faris Wirakusuma	Implementasi popup, styling pada ui, dan content, modifikasi pada vite config, perbaikan pada ocr dan mengerjakan laporan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Reynard Anderson Wijaya



Ega Luthfi Rais



Faris Wirakusuma Triawan

Tautan repository Github: https://github.com/road2qnt/tubes3_PULAU777

Daftar Pustaka

- **String Pattern Searching**
<https://www.geeksforgeeks.org/dsa/introduction-to-pattern-searching/>
- **Chrome Extension Documentation**
<https://developer.chrome.com/docs/extensions/>
- **Tesseract.js**
<https://tesseract.projectnaptha.com/>
- **Mozilla Developer Network**
<https://developer.mozilla.org/en-US/docs/Web/API/Document/createTreeWalker>